10c_c3_BLOCKED_red_and_plan

10c — C3 (registry seed-only / fail-closed resolver): BLOCKED handoff + RED test + implementation plan

Date: 2026-07-06 **Status: BLOCKED** — C3's acceptance criterion depends on unlanded infra (C4 + C5) and an under-specified resolver DB data-source. Per §11.4.6 (no-guessing) and §11.4.101, the fail-closed resolver is NOT implemented on a guess. This note carries the §11.4.115 RED reproduce-first test (with captured RED→GREEN evidence) + a precise file:line plan so a future executor lands C3 after C4/C5. **Scope target:** `submodules/llms_verifier/llm-verifier/capabilities/registry.go` (module `digital.vasic.llmsverifier`), branch `feature/helixllm-full-extension`. **Spec:** `docs/research/07.2026/10_llmsverifier_helixagent/10b_code_exact_change_spec.md` §3 C3 + §2.5 + §4.

---

1. Why C3 is BLOCKED (exact ambiguity + infra gap)

C3 = "demote `registry.go` static block: seed-only, probe-overridden, fail-closed" (10b §3 C3). It has 4 parts; its **acceptance criterion** is parts 2+3 — a resolver that **prefers a fresh `database.VerificationResult`** (per model, within the CONST-037 24h / CONST-038 60s window using `VerificationResult.StartedAt/CompletedAt`) and **fail-closes to `unverified`** when no probe exists. Two hard blockers:

1. **Depends on unlanded C4 + C5 (spec's own ordering).** 10b §4.3 states verbatim: *"C4 before C3's fail-closed resolver and before C5 — probes are the source of truth the resolver prefers and the service dispatches."* Verified this session:
   - **C5 NOT landed** — `verification/verification.go:62` still `return nil, ErrVerificationNotWired`.
   - **C4 NOT landed** — there is no per-capability probe path writing per-model `database.VerificationResult` rows through the capability surface (only C1's downstream `testParallelToolUse` tool-call counting exists at `llmverifier/verifier.go:1112`). With no producer of fresh per-model `VerificationResults`, the resolver's *primary* branch ("prefer fresh probe") is not exercisable end-to-end and

would ship as an unwired path (§11.4.124 dead-code hazard) with a mock-only test — exactly why the spec orders C4/C5 first.

2. **Resolver DB data-source unspecified at file:line (§11.4.6 guess forbidden).** The existing accessors are **provider-keyed and DB-handle-less**:
   - `GetProviderBaseCapabilities(provider string) *ProviderCapabilities` — `registry.go:1068`
   - `GetProvidersWithCapability(capName string, capValue interface{}) []string` — `registry.go:1102` To "prefer a fresh `database.VerificationResult` **(per model)**" the resolver needs (a) a `*database.Database` handle, (b) a **per-model** identity (accessors are per-**provider**), and
   1. a latest-per-model query. The spec does NOT pin how the accessors acquire the handle (global var? new resolver struct? added param breaking all callers?). Import direction is feasible (`database` does NOT import `capabilities`, so no cycle) — but the design choice is left open and MUST NOT be guessed.

Not-blocked facts (for the executor): `capabilities` package builds + tests green today; the only in-repo callers of `GetProviderBaseCapabilities` are 3 in-package sites (`capabilities/detector.go:48,153,292`); `GetProvidersWithCapability` has no in-repo callers. `ProviderCapabilities.Verified` is the hand-authored self-certification literal (openai `registry.go:11` `Verified: true`).

---

2. §11.4.115 RED reproduce-first test (captured evidence)

The RED test below was created at `capabilities/registry_c3_failclosed_red_test.go`, run in both polarities, and then REMOVED (BLOCKED — not reddening the baseline; the source lives here as the handoff artefact). Oracle = the PUBLIC accessor's returned `.Verified` absent any probe. Compiles on the current pre-C3 accessor signature (references only the existing `.Verified` field).

```go
package capabilities

import (
    "os"
    "testing"
)

// TestC3RegistrySeedNotSelfCertifiedVerified — §11.4.115 RED-baseline for C3
```

```go
// (10b §3 C3, §2.5). Gap: GetProviderBaseCapabilities("openai")
        returns a seed
// self-certified .Verified==true (registry.go:11) with NO probe
        backing.
// Post-C3, absent a fresh probe the accessor MUST report
        unverified/fail-closed.
func TestC3RegistrySeedNotSelfCertifiedVerified(t *testing.T) {
    redMode := os.Getenv("RED_MODE")
    if redMode == "" {
        redMode = "0"
    }
    caps := GetProviderBaseCapabilities("openai")
    if caps == nil {
        t.Fatalf("GetProviderBaseCapabilities(\"openai\")
        returned nil; expected a seed entry")
    }
    selfCertifiedVerified := caps.Verified // no probe supplied
    switch redMode {
    case "1": // reproduce: PASSes on broken (current) artifact
        if !selfCertifiedVerified {
            t.Fatalf("RED_MODE=1: expected seed self-certified
        Verified==true absent a probe, got %v", caps.Verified)
        }
        t.Logf("RED_MODE=1 PASS: defect reproduced — unbacked
        seed self-certified Verified==true (§2.5).")
    case "0": // guard: FAILs on broken, PASSes once fail-closed
        C3 lands
        if selfCertifiedVerified {
            t.Fatalf("RED_MODE=0: fail-closed C3 not implemented
        — unbacked seed self-certified Verified==true; " +
                "absent a fresh probe the accessor MUST report
        unverified/fail-closed (10b §3 C3 part 3).")
        }
        t.Logf("RED_MODE=0 PASS: accessor reports unverified/
        fail-closed absent a probe.")
    default:
        t.Fatalf("unknown RED_MODE=%q (expected 0 or 1)",
        redMode)
    }
}
```

Captured evidence (this session, `cd llm-verifier`):

```
===== RED_MODE=0 (would-be-fixed guard) — FAIL (RED proof: C3 gap is real)
--- FAIL: TestC3RegistrySeedNotSelfCertifiedVerified (0.00s)
    registry_c3_failclosed_red_test.go:58: RED_MODE=0: fail-closed C3 not
```

```
    GetProviderBaseCapabilities("openai") returns an unbacked seed self-ce
    absent a fresh probe the accessor MUST report unverified/fail-closed (
FAIL    digital.vasic.llmsverifier/capabilities 0.004s   EXIT=1

===== RED_MODE=1 (reproduce defect) — PASS (defect reproduced) =====
--- PASS: TestC3RegistrySeedNotSelfCertifiedVerified (0.00s)
    RED_MODE=1 PASS: defect reproduced — GetProviderBaseCapabilities("open
    hand-authored seed self-certified Verified==true with NO probe backing
ok      digital.vasic.llmsverifier/capabilities 0.003s   EXIT=0
```

RED_MODE=0 FAILs on the current artifact → the C3 gap is genuinely present and unfixed. Baseline restored after capture: `go build ./...` EXIT=0, `go test ./capabilities/...` = ok, working tree clean.

---

3. Precise file:line implementation plan (execute AFTER C4 + C5)

1. **Rename** + **demote the seed map** — `registry.go:8` `var providerCapabilities` → `var providerCapabilitySeeds`, with a doc-comment: “hand-authored bootstrap defaults; NOT verified — MUST be overridden by a live probe / DB `VerificationResult` before being shown to any user, per CONST-036/037/040.” Update the 4 internal references (`registry.go:1069,1086,1105` and the `GetAllProviders` loop). Set every seed’s `Verified` literal to `false` (a seed is unverified by construction — this alone flips the RED_MODE=0 guard GREEN and satisfies §2.5 part 1).
2. **Add a resolver** (new `registry_resolve.go`, same package) with an explicit contract — decide the DB data-source WITH the operator or as part of C5’s wiring (do NOT guess): e.g. `func ResolveModelCapability(db *database.Database, provider, model, capName string) (value bool, verified bool, err error)` that
    1. loads the latest `database.VerificationResult` for `model`; (b) if fresh (StartedAt/CompletedAt within CONST-037 24h / CONST-038 60s) returns the probed value + `verified=true`; (c) else if a seed exists returns the seed value + `verified=false`; (d) else fail-closed (`false, false, nil`) → unverified, NEVER a silent hand-authored literal. `database` is safe to import (no cycle).
3. **Route accessors** `GetProviderBaseCapabilities:1068` / `GetProvidersWithCapability:1101` through the resolver (reconcile the 3 `capabilities/detector.go` callers + signatures per §11.4.120).
4. **Demote static literals** (e.g. anthropic `FunctionCalling:false` `registry.go:102`) to seeds; the C4 probe is the source of truth.
5. **Re-land the RED test** from §2 into `capabilities/`, flip default polarity to the standing GREEN guard (`RED_MODE=0`), register it in the §11.4.135

regression-guard suite. Add per-capability fail-closed cases (§11.4.146 STEP 3 extend).

Dependency order (10b §4): C4 → C5 → C3. Do NOT land the resolver's "prefer fresh probe" branch before C4/C5 produce real per-model `VerificationResults`, or it ships unwired (§11.4.124).

Sources verified

- Code read directly this session (2026-07-06) from `submodules/llms_verifier/llm-verifier/...`: `capabilities/registry.go:8,1068,1102`, `capabilities/detector.go:48,153,292`, `verification/verification.go:62,69`, `llmverifier/verifier.go:1112`.
- Spec: `10b_code_exact_change_spec.md` §2.5, §3 C3, §4.
- RED evidence captured this session (temp test run + removed).